

Reviewer Pack — Coxeter Group Action on Boolean Function Entropy via Invaria...

Entry 1fce3364b200 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 22:22:59 UTC

Coxeter Group Action on Boolean Function Entropy via Invariant Generators

Entry ID: 1fce3364b200 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 22:22:59 UTC

1. Conjecture

Field A (mathematical branch): Coxeter group theory **Field B** (complexity object): Boolean function entropy

Statement:

For a given Boolean function f with n variables, the number of distinct minimal generators for the Coxeter group action on the set of its homomorphisms is $O(n^{1/3})$. Equivalently, the entropy rate $H(f)$ of f satisfies $|H(f)| \leq k * n^{1/3}$, where k is an absolute constant.

Rationale (proposer's reasoning):

Coxeter groups have been used to study symmetry in various mathematical contexts. Their action on homomorphisms of a Boolean function could potentially reveal structural information about the function's complexity, such as its entropy. The connection between Coxeter group theory and entropy rates provides a novel way to explore the complexity of Boolean functions.

Taxonomy category: Coxeter_group_action_on_boolean_function_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 66fb26b269106229

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The number of minimal generators for the Coxeter group action on homomorphisms of Boolean function f with n variables is $O(n^{1/3})$, and the entropy rate $|H(f)|$ is within a factor k of $n^{1/3}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group theory" AND "Boolean function entropy" - "minimal generators" AND "Coxeter group action" ON "Boolean functions" - "entropy rate" AND " $n^{1/3}$ " IN "Boolean functions"

Top relevant hits considered: - [s2:1808.01137] When Does Hillclimbing Fail on Monotone Functions: An entropy compression argument - [s2:2311.07438] Hardest Monotone Functions for Evolutionary Algorithms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def hamming_distance(x, y):
    return sum(xi != yi for xi, yi in zip(bin(x)[2:].zfill(n), bin(y)[2:].zfill(n)))

def generate_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(1 << n)]

def min_generators_count(f):
    n = int(math.log2(len(f)))
    hom = [f[hamming_distance(j, i)] for j in range(1 << n)]
    generators = set()
    for i in range(1 << n):
        if all(hom[i] != f[j] for j in range(i)):
            generators.add(i)
    return len(generators)

def entropy_rate(f):
    n = int(math.log2(len(f)))
    counts = [f.count(v) for v in set(f)]
    probabilities = [Fraction(c, 1 << n) for c in counts]
    return -sum(p * math.log2(p) if p != 0 else 0 for p in probabilities)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        f = generate_boolean_function(n)
```

```

        generators_count = min_generators_count(f)
        H_f = entropy_rate(f)
        results.append({
            "n": n,
            "generators_count": generators_count,
            "H_f": H_f
        })
    mean_generators_count = sum(r["generators_count"] for r in results) / len(results)
    mean_H_f = sum(r["H_f"] for r in results) / len(results)
    conjecture_holds = all(abs(generators_count - n**(1/3)) <= 10 * n**(1/3) for r in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "generators_count",
        "metric_value": mean_generators_count,
        "instances_tested": len(results),
        "n_max": max(r["n"] for r in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c126ee59.py", line 69, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c126ee59.py", line 44, in run_trial
    generators_count = min_generators_count(f)
    ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c126ee59.py", line 26, in min_generators_count
    hom = [f[hamming_distance(j, i)] for j in range(1 << n)]

```

UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Debug the test code to identify and fix the crash, then rerun the test to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22995
2	preregistration	ollama_remote	glm4:latest	0	0	12077
3	novelty	ollama_remote	glm4:latest	0	0	16010
4	novelty	ollama_remote	glm4:latest	0	0	14684
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13251
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21003
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14219
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23580
9	judge	ollama_remote	glm4:latest	0	0	8882

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146701 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1fce3364b200.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1fce3364b200.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1fce3364b200.tar.gz` (if generated)